

FIAP · Pós Tech · Arquitetura e Desenvolvimento Java
Tech Challenge — Fase 2

Sistema de Biblioteca Online

Relatório Técnico

Autor: Mateus S. Abreu

Repositório: github.com/Nexusf1re/Biblioteca-FIAP

Data: Junho de 2026

1. Visão geral

A proposta desse Tech Challenge era construir uma API REST para uma biblioteca online. Na prática, o sistema permite cadastrar livros e usuários, registrar empréstimos e devoluções, fazer reservas de livros que estão indisponíveis e tirar dois relatórios de uso: os 20 livros mais emprestados e a lista do que está emprestado no momento com a previsão de devolução.

Tentei não complicar mais do que o necessário, mas também deixar uma base que aguentasse crescer: o código está separado em camadas, o banco é relacional, as listagens são paginadas e tudo roda em container. Nas próximas seções comento as escolhas que fiz e, principalmente, os perrengues que apareceram no caminho.

2. Tecnologias e ferramentas

Ferramenta	Pra que serviu
Java 21 (LTS)	Versão LTS atual. Usei bastante Streams e a API de data/hora (<code>java.time</code>).
Spring Boot 3.3	Web, injeção de dependência, validação e acesso a dados sem muito boilerplate.
Spring Data JPA / Hibernate	Mapeamento das entidades e repositórios prontos, já com paginação.
PostgreSQL 16	Banco "de verdade", rodando em container.
H2	Banco em memória pra desenvolver e rodar os testes rápido.
springdoc-openapi	Swagger gerado a partir do próprio código.
Docker / Docker Compose	Empacotar a aplicação e o banco e subir tudo junto.
Maven	Build e dependências. Deixei o wrapper (<code>mvnw</code>) no repositório.
JUnit 5 / AssertJ / Testcontainers	Testes unitários, de integração e um de regressão com Postgres real.

3. Arquitetura e organização do código

Segui uma arquitetura em camadas, que é o feijão com arroz mas funciona bem: o fluxo de uma requisição passa por `controller` → `service` → `repository` → banco . O controller cuida do HTTP e dos DTOs, o service guarda as regras de negócio e as transações, e o repository fala com o banco via Spring Data.

Uma coisa que fiz questão de manter foi não deixar entidade vazar pra fora: os controllers só trabalham com DTOs (que fiz como `record`). Isso separa o modelo interno do contrato da API, então dá pra mexer nas tabelas sem necessariamente quebrar quem consome. Os erros ficam todos num tratador global, pra resposta sair sempre no mesmo formato.

Os pacotes ficaram divididos assim:

- `controller` , `service` , `repository` — as três camadas.

- `domain` — as entidades (Livro, Usuário, Empréstimo, Reserva) e os enums.
- `dto` — entradas e saídas da API.
- `exception` — exceções de domínio e o `@RestControllerAdvice`.

4. Modelo de dados e regras de negócio

São quatro entidades. O ponto que mais pensei foi o Livro: em vez de um campo "disponível" booleano, ele guarda quantos exemplares existem e quantos estão livres. Confesso que no começo eu tinha feito como booleano, e foi só quando fui pensar nas reservas e no relatório que percebi que não ia dar — voltei atrás e modeliei por quantidade. Com isso dá pra ter várias cópias do mesmo título, que é o que faz sentido numa biblioteca.

O resto:

- **Usuário** — nome, e-mail (único) e tipo (ALUNO, PROFESSOR ou MEMBRO). O histórico de empréstimos é só uma consulta nos empréstimos dele.
- **Empréstimo** — liga livro e usuário, guarda a data do empréstimo (instante em UTC), a prevista e a real de devolução, e o status.
- **Reserva** — entra quando não tem exemplar livre, numa fila por livro.

As regras principais que implementei:

- Nada de ISBN ou e-mail repetido.
- Empréstimo só rola se tiver exemplar; ao emprestar, a disponibilidade cai 1 e a previsão vira `hoje + 14 dias` (em UTC, e dá pra configurar).
- Na devolução o exemplar volta e a primeira reserva da fila daquele livro é atendida sozinha.
- Não deixo excluir livro nem usuário que ainda tenha empréstimo em aberto.
- O atraso eu não guardo no banco — calculo na hora, comparando a data prevista com hoje. Achei mais simples do que ficar com um campo que precisaria de um job pra atualizar.

5. Decisões de design

- **Tudo em UTC.** Defini cedo que o sistema inteiro trabalha em UTC e as datas saem em ISO-8601. Pode parecer detalhe, mas evita aquele bug chato de "um dia a mais/a menos" dependendo de onde a aplicação roda, e deixou o cálculo dos prazos previsível.
- **Schema pelo Hibernate (`ddl-auto`) + índices nas anotações.** Pro escopo do desafio, deixar o Hibernate gerar o schema simplificou muito. Sendo bem honesto, se fosse pra produção eu teria ido de Flyway desde o começo pra versionar o banco — fica anotado como próximo passo.
- **DTOs como record e sem Lombok.** Menos dependência e menos mágica.
- **Erro centralizado.** Um advice traduz as exceções pra 400 (validação), 404 (não achou) e 409 (conflito de regra), sempre no mesmo corpo de resposta.

6. Otimização de acesso a dados

Esse era um dos pontos pedidos, então cuidei de alguns detalhes:

- Paginação em todas as listagens, com um envelope próprio pra não amarrar a API à estrutura interna do `Page` do Spring.
- Os relatórios são agregados direto no banco (`GROUP BY / COUNT`), sem puxar tudo pra memória pra contar.
- `JOIN FETCH` no relatório de empréstimos abertos, pra fugir do N+1.
- Índices nos campos que mais aparecem nas buscas (ISBN, e-mail, status e as FKs).
- Cadastro em lote (`POST /livros/lote`) com `batch_size` configurado.
- `@Version` no livro, pra dois empréstimos ao mesmo tempo não pegarem o mesmo último exemplar.

7. Docker e escalabilidade

A parte de Docker eu quis deixar redonda, porque é bastante cobrada. O Dockerfile é multi-stage: um estágio compila com Maven e o outro, bem menor, roda só com a JRE e em usuário não-root.

O Compose sobe a aplicação e o Postgres juntos. Uma coisa que precisei ajustar: no começo a API subia antes do banco estar pronto e quebrava. Resolvi com `healthcheck` no Postgres e `depends_on: condition: service_healthy`, então a API só inicia depois que o banco responde. Tem volume pra não perder os dados e a configuração toda vem por variável de ambiente, sem nada sensível fixo no código.

Como a API é stateless (não guarda nada em memória entre requisições), dá pra rodar várias instâncias com `docker compose up --scale app=N` atrás de um balanceador, sem mudar código. Foi pensando nisso que evitei guardar estado na aplicação.

8. Desafios encontrados e soluções

8.1. O bug que só aparecia no PostgreSQL

Esse me rendeu um tempo. Durante o desenvolvimento eu testava no H2 e estava tudo verde. Quando subi a stack real no Docker e fui testar a busca de livros, tomei um 500 na cara:

```
ERROR: function lower(bytea) does not exist
```

Demorei pra entender. O que acontecia: na busca por filtros opcionais, quando um filtro vinha nulo, o PostgreSQL não conseguia descobrir o tipo daquele parâmetro dentro do `LOWER(...)` e chutava `bytea` — e não existe `lower` pra `bytea`. O H2 é mais relaxado e simplesmente deixava passar, por isso nunca tinha estourado nos testes locais.

A correção foi tipar o parâmetro na consulta com `CAST(:parametro AS string)`. Depois disso passou a funcionar igual nos dois bancos.

Ficou a lição de que H2 e PostgreSQL não se comportam igual, e que validar só no H2 dá uma falsa sensação de segurança. Depois desse susto montei um teste de regressão com Testcontainers, que sobe um Postgres de verdade só pra esse caso não voltar a passar batido.

8.2. Datas e fuso

Cálculo de prazo é sensível a fuso, então padronizar tudo em UTC desde o início me poupou de bug bobo. Foi mais uma decisão preventiva do que uma dor — mas é o tipo de coisa que, se eu deixasse pra resolver no fim, ia dar retrabalho na camada de datas inteira.

8.3. Testcontainers contra um Docker muito novo

Na minha máquina a versão do Docker era nova demais e o cliente do Testcontainers não conseguia conversar com ela (dava erro 400 na negociação). Não quis travar o build por causa disso, então deixei o teste com `@Testcontainers(disabledWithoutDocker = true)` : ele roda onde o Docker é compatível e se desliga sozinho onde não é. Enquanto isso, a validação contra o Postgres eu fiz na mão com o Docker Compose.

9. Testes

Foquei no que tinha mais risco de quebrar:

- Testes unitários da disponibilidade do livro (emprestar/devolver exemplar).
- Um teste que sobe o contexto inteiro, só pra garantir que nada está mal configurado.
- Testes de integração do fluxo completo: empresta, devolve, a reserva sendo atendida na devolução, o prazo de 14 dias e os relatórios.
- O teste de regressão com Testcontainers da busca (o tal do `lower(bytea)`).

Além disso rodei a aplicação no Docker e fui batendo nos endpoints na mão pra conferir o CRUD, o empréstimo/devolução, as reservas, os relatórios e os códigos de erro (400/404/409).

10. Como dá pra escalar

O sistema começa pequeno, mas alguns caminhos de evolução já estão claros:

- Subir mais instâncias da API atrás de um balanceador (ela já é stateless), e mais pra frente orquestrar com Kubernetes.
- Réplicas de leitura no Postgres pra distribuir as consultas dos relatórios e buscas.
- Um cache (Redis, por exemplo) pros relatórios mais acessados.
- Migrar o schema pra Flyway/Liquibase, pra evoluir o banco com segurança.
- Uma fila pra operações em lote mais pesadas, tipo importação de acervo.
- Observabilidade: o Actuator já está ligado, daria pra somar logs centralizados e tracing.

11. Documentação da API

A documentação sai automática pelo springdoc, em `/swagger-ui.html` (o contrato OpenAPI fica em `/v3/api-docs`). De lá dá pra ver os endpoints, os parâmetros e até testar as chamadas. O código também tem comentários que podem virar Javadoc.

12. Conclusão

No fim o sistema entrega tudo que foi pedido — CRUD de livros e usuários, empréstimo e devolução com cálculo de prazo, busca com filtros, reservas e os dois relatórios — usando as APIs modernas do Java, banco relacional com as consultas otimizadas e Docker pra subir tudo.

Mais do que cumprir a lista de requisitos, o que mais me marcou foi o aprendizado do meio do caminho, tipo a diferença de comportamento entre H2 e Postgres e o quanto vale testar contra o ambiente real. Se eu fosse continuar o projeto, começaria pelo Flyway e por mais cobertura de testes — mas, pro que o desafio pedia, acho que ficou bem resolvido.